

1.) What is the React ?

- => React is an open-source JavaScript library for building user interfaces
- => Component-Based Structure:
- => Virtual DOM:
- => One-Way Data Flow: (from parent to child components)
- => JSX (JavaScript XML):

2.) What are the key feature of React ?

- => **Component-Based Architecture:** (React organizes the UI into small, reusable components, each managing its own state.)
- => **Virtual DOM:** (React uses a virtual DOM to update only the parts of the UI that have changed, rather than re-rendering the entire DOM.)
- => **One-Way Data Binding:** (React follows a unidirectional data flow, making the data more predictable and easier to debug. This structure helps in managing the state and logic within the application.)
- => **JSX (JavaScript XML):** (JSX is a syntax extension that allows you to write HTML-like code directly in JavaScript.)
- => **Declarative UI:** (React follows a declarative paradigm, where you describe the desired state of your UI, and React handles the updates automatically.)
- => **State Management:** (React's built-in useState and useReducer hooks, along with Context API, help manage application state.)
- => **Hooks:** Introduced in React 16.8, hooks allow you to use state and lifecycle features in functional components, which were previously only available in class components.
- => **React Router:** Though not a part of React itself, React Router is often used with it for managing navigation and routes within single-page applications
- => **Cross-Platform Development:** With React Native, React code can be used to build mobile applications for iOS and Android using the same concepts and components.

3.) Explain the differences between Real Dom and Virtual Dom

Real Dom

- Actual representation in the browser - Nature -
- Updates immediately with every change - Update Approach -
- Slower, especially for frequent updates - Performance -
- Re-renders all affected nodes - Re-rendering -
- Managed by the browser - Usage -

Virtual Dom

- In-memory copy of the Real DOM - Nature -
- Updates in batches by comparing changes - Update Approach -
- Faster, by minimizing real DOM updates - Performance -
- Only re-renders changed nodes - Re-rendering -
- Managed by React for optimizing updates - Usage-

4.) What are the Limitation of react ?

While React is powerful, it also has some limitations and challenges:

- a) Learning Curve for Beginners
- b) Complex State Management in Large Applications
- c) SEO Challenges
- d) Heavy Initial Setup for Complex Apps
- e) High Pace of Development
- f) Limited Built-In Functionality
- g) Poor Support for Older Browsers
- h) Performance Issues with Large Applications
- i) JSX as a Barrier for Designers
- j) Memory Usage for Virtual DOM

5.) What is the JSX ?

JSX (JavaScript XML) is a syntax extension for JavaScript that resembles HTML. It's used in React to describe what the UI should look like. JSX allows developers to write HTML-like code directly within JavaScript, which React then compiles into JavaScript code to create and manipulate the DOM.

6.) What are the component ?

Components are the fundamental building blocks of React applications. They are reusable pieces of code that encapsulate a specific part of the UI, making your code more organized, modular, and easier to maintain.

1. Functional Components:

Defined as simple JavaScript functions that take props as input and return a JSX element representing the component's UI.

Typically used for stateless components that don't require internal state.

2. Class Components:

Defined using JavaScript classes that extend the `React.Component` class.

Can have state, lifecycle methods, and other features for more complex components.

3. Stateless Functional Components (SFCs):

A special type of functional component that is optimized for performance.

They are often preferred over class components for simple, stateless components.

4. Higher-Order Components (HOCs):

Functions that take a component as input and return a new component with enhanced capabilities.

Used for common patterns like data fetching, error handling, and conditional rendering.

5. Pure Components:

A type of component that performs shallow comparison of props and state to determine if a re-render is necessary.

Can improve performance by avoiding unnecessary re-renders.

6. Custom Hooks:

Reusable functions that encapsulate state and logic within functional components.

Provide a way to share custom logic between components without creating class components.

7.) Why Browser can't read the JSX ?

Browsers cannot directly read JSX because it's a syntax extension for JavaScript, not a standard language feature. JSX is a more readable way to define UI elements in JavaScript, but browsers only understand plain JavaScript.

8.) What are the difference between the class component and functional component ?

A.) Class-Based-Component

- => Extend `React.Component`
- => Have their own state using `this.state`
- => Have a built in lifecycle method
- => Typically require the constructor
- => Can receive props from the parent components

B.) Functional Component

- => Regular Javascript component
- => No internal state
- => No Lifecycle method
- => No constructor needed
- => Receive props from the parent components

9.) What are the difference between uncontrolled and controlled component

Controlled Components

- => Directly managed by the parent component
- => Receives value prop
- => Parent component's state
- => Parent component handles changes

Uncontrolled Components

- => Managed by their own internal state
- => doesn't receive a value prop from the parent.
- => Controlled by internal state
- => The component listens for change events and updates its own state.

10.) what is the difference between the state and props ?

- => State and props are two fundamental concepts in React components that serve different purposes:

STATE

- => State is used to store data that belongs to a specific component and can change over time. It's like the component's memory.
- => When the state of a component changes, React re-renders the component to reflect the updated data.

=> State is declared and managed within the component itself using the `this.state` object in class components or the `useState` hook in functional components.

PROPS

=> Props are data passed down from a parent component to a child component. They're like arguments to a function.

=> Props are generally considered immutable within the child component. If you need to modify them, you should update the parent component's state.

=> Props are used for communication between components, allowing data to flow from parent to child.



11.) How do `useState` and `useEffect` works in react ?

`useState`:

=> used to manage state within function components

Takes an initial state value and returns a pair of values: the current state and a function to update the state.

`useEffect`:

=> `useEffect` runs asynchronously after the browser has painted. Won't block the browser from updating and loading DOM

=> `useEffect` hooks is used to fetch the data from API and update the components state.

=> the empty dependency array `[]` indicates that the effect should run only once after the component is mounted.

=> if you want the effect to run whenever a dependency changes, include those dependencies in array

12.) what is the important key in react ?

=> Components:

=> JSX

=> Virtual DOM

=> State and props

=> Hooks

=> Lifecycle methods

=> Conditional Rendering

=> Lists and Keys

=> Forms

=> Event handling

=> Context API

=> Fragments

=> Portals

=> Higher Order Components

13.) explain the concept of lifting the state up in react ?

=> Lifting State Up is a common pattern in React where you move shared state and logic to a common ancestor component that can manage it for its descendants.

=> By understanding and applying the concept of lifting state up, you can create more organized, maintainable, and scalable React applications.

14) What are the Hooks in React? Name of common hooks

=> Hooks are functions that let you use React features like state and lifecycle methods without writing a class component.

COMMON HOOKS

1. useState:

=> Used to manage state within functional components.

Takes an initial state value and returns a pair of values: the current state and a function to update the state.

For managing simple state values.

2. useEffect:

=> Used to perform side effects in functional components, such as data fetching, subscriptions, or DOM manipulations.

Takes a callback function and an optional dependency array. The callback function is executed after the component is rendered and whenever the dependencies change.

3. useContext:

=> used to access values from a context provider without passing props through intermediate components

4. useReducer:

=> An alternative to useState for managing complex state logic. It takes a reducer function and an initial state value.

5. useRef:

=> Used to create a mutable ref object that can be accessed from a functional component.

6. useCallback:

=> memorizes a callback function, preventing unnecessary re-renders if its dependencies haven't changed.

7. useMemo:

=> memorize a value, preventing unnecessary re-calculations if its dependencies haven't changed.

8. useImperativeHandle:

=> Exposes a custom ref to a parent component, allowing the parent to access or manipulate the child component's internal state or methods.

9. useEffect:

=> Similar to useEffect but runs after DOM mutations.

=> useLayoutEffect runs synchronously its block browser visual update until its callback has completed

10. useTransition:

=> Helps manage state transitions that might cause visual glitches it returns a startTransition function and a isPending boolean.

15.) How does one pass data between components in React ?

=> Passing data between components in React is a fundamental aspect of building complex applications. There are primarily two methods to achieve this:

1. Props

2. Context API:

16.) What are the new features introduced in React-18

1. Suspense:

=> Data fetching and rendering: Suspense allows components to declaratively wait for data to be fetched before rendering, improving user experience and reducing complexity.

=> Concurrent rendering: Suspense enables concurrent rendering, where multiple components can render and update simultaneously, leading to better perceived performance.

2. Automatic batching:

=> Optimized updates: React now automatically batches state updates within the single render pass, reducing unnecessary re-renders and improving the performance.

3. Client-side rendering (CSR) improvements:

=> Faster initial render : React 18 has optimization for faster initial rendering, especially in large application

=> Streaming: Improved streaming capabilities allow for progressive rendering of content, providing the better experience.

4. Server-side rendering (SSR) improvements:

=> faster initial page load : react 18 has enhancements for faster SSR, leading to quicker initial page loads.

=> Streaming SSR: Improved streaming support for SSR allows for progressive rendering on the server side, enhancing performance.

5. New Hooks :

=> useTransition: Provides a way to manage state transitions that might cause visual glitches.

=> useDeferredValue : Defer updates to a value until the browser is less busy improving perceived performance.

=> useSyncExternalStore : allow for more efficient integration with external state management.

6. Strict Mode:

=> improved code quality : Strict mode helps identify potential issues in your code, such as missing dependencies or incorrect lifecycle methods.

7. Component updates:

=> More granular updates : React 18 provides more granular control over component updates, allowing for fine-tuning performance and behavior.

17.) What is concurrent rendering in React 18 ?

=> Concurrent Rendering is a significant feature introduced in React 18 that enables multiple components to render and update simultaneously, improving perceived performance and user experience.

18.) How does automatic batching work in React-18 ?

=> Automatic batching is a significant performance optimization introduced in React 18. It aims to reduce the number of unnecessary re-renders by grouping multiple state updates within a single render pass.

19.) Explain the useDeferredValue in React 18 ?

=> useDeferredValue is a React hook introduced in React 18 that helps improve perceived performance by deferring updates to a value until the browser is less busy. This is particularly useful for non-critical updates that can be delayed without affecting the user experience.

20.) What is the useTransition hooks in react 18

=> useTransition hooks introduced in version 18 that helps manage state transitions that might cause visual glitches or performance issues.

21.) What is the suspense and how does it work ?

=> Suspense is the new feature introduced in React that allows components declaratively wait for data to be fetched before rendering. This improves user experience by preventing blank screens or loading spinners while data is being fetched.

Suspense is a declarative way to handle data fetching and loading states.

It improves user experience by providing a more seamless transition between loading and rendered content.

You can use Suspense with any data fetching mechanism.

It's important to handle errors and provide appropriate fallback content.

Concurrent Rendering: Suspense is tightly integrated with React's new concurrent rendering feature.

22.) How has React 18 improved suspense ?

1. Concurrent Rendering:

=> Suspense is tightly integrated with React's new concurrent rendering feature.

2. Fallback UI:

=> React 18 introduces a new fallback prop for Suspense components.

3. Error boundaries:

=> Suspense can be used in conjunction with error boundaries to catch errors that occur during data fetching or rendering.

4. Improved streaming:

=> React 18 has improved streaming capabilities for both client-side rendering (CSR) and server-side rendering (SSR).

5. Integration with other features:

=> Suspense works seamlessly with other React features like hooks, context, and reducers,

23.) What is the new startTransition function in React ?

=> startTransition allows you to defer updates to a state value until the browser is less busy, improving perceived performance and reducing visual glitches.

=> By deferring updates, startTransition can improve the perceived performance of your application.

=> You can use the pending value to provide a loading indicator or other visual feedback while the transition is in progress.